Software Architecture and Design Specification

Project: Traffic Management System (Simulation)

Version: 1.0

Authors: P.Harini, Neal Punjabi

Date: 03-10-2026

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	03/10/2026	P.Harini, Neal Punjabi	Initial SAD Document

Approvals

Role	Name	Signature/Date

1. Introduction

1.1 Purpose

This Software Architecture and Design Specification (SAD) outlines the architecture and design of the Traffic Management System (Simulation).The system is a C-based standalone traffic simulation which simulates vehicle arrivals, vehicle queues, traffic signals, vehicle movement, traffic density, signal timing and basic traffic statistics.

The system primarily uses queue/FIFO data structures to represent waiting vehicles on different directions of an intersection. It has logical modules for vehicle generation, queue management, signal management, configuration, simulation control, and reporting.

The system is designed for educational and analytical purposes and should not be used for direct traffic signal control or to control real-world vehicles, cameras, sensors, GPS systems, or IoT traffic-control devices.

1.2 Scope

The SAD covers the architecture and design of the following major functions:

- Vehicle generation and arrival simulation
- Queue management (FIFO)
- Traffic signal control
- Traffic density simulation
- Signal timing configuration
- Vehicle movement
- Queue status monitoring
- Traffic statistics and reporting
- Simulation configuration
- Start, pause/resume and stop operations
- Console based user interaction
- Input validation and error handling
- Safe queue and memory operations

The system is implemented in C using queues, structures, arrays and (where required) linked lists.

1.3 Audience

The document is intended for the following audiences:

- Project Developers – to understand the system architecture and module design.
- Project Guide/Faculty – to evaluate the architecture and design.
- Testers – to understand module interactions and expected system behavior.
- Students – to understand the application of queues and other data structures.
- Future Maintainers – to understand the organization and dependencies of the system.

1.4 Definitions

Term	Meaning
TMS	Traffic Management System
SAD	Software Architecture and Design Specification
SRS	Software Requirements Specification
UML	Unified Modeling Language
FIFO	First-In-First-Out
UI	User Interface
CPU	Central Processing Unit
RAM	Random Access Memory
OS	Operating System
Queue	Data structure used to maintain waiting vehicles
Enqueue	Adding a vehicle to the rear of a queue
Dequeue	Removing a vehicle from the front of a queue

2. Document Overview

2.1 How to use this document

This document describes the architecture and design of the Traffic Management System.

It provides:

- System architecture
- Major software components/modules
- Component responsibilities
- Architecture pattern and rationale
- Technology stack
- Data structures
- Module interactions

- UML component diagram
- UML sequence diagrams
- Error handling
- Security design
- Traceability between requirements and design

The SAD should be used together with the SRS and Test Plan during implementation, testing and maintenance.

2.2 Related Documents

The following documents are related to this SAD:

1. **Software Requirements Specification (SRS)** – defines the functional, non-functional and security requirements.
2. **Software Test Plan** – defines testing strategy, test cases and acceptance criteria.
3. **Requirements Traceability Matrix (RTM)** – maps applicable requirements to corresponding design and test elements.

3. Architecture

3.1 Goals & Constraints

Architecture Goals

The architecture is designed to:

- Provide a simple and modular traffic simulation.
- Separate different system responsibilities.
- Efficiently manage waiting vehicles using FIFO queues.
- Correctly simulate Red, Yellow and Green signal states.
- Allow traffic density and signal timing configuration.
- Maintain consistent vehicle and queue information.
- Provide traffic statistics.
- Handle invalid inputs safely.
- Support easy testing and maintenance.

Constraints

The architecture has the following constraints:

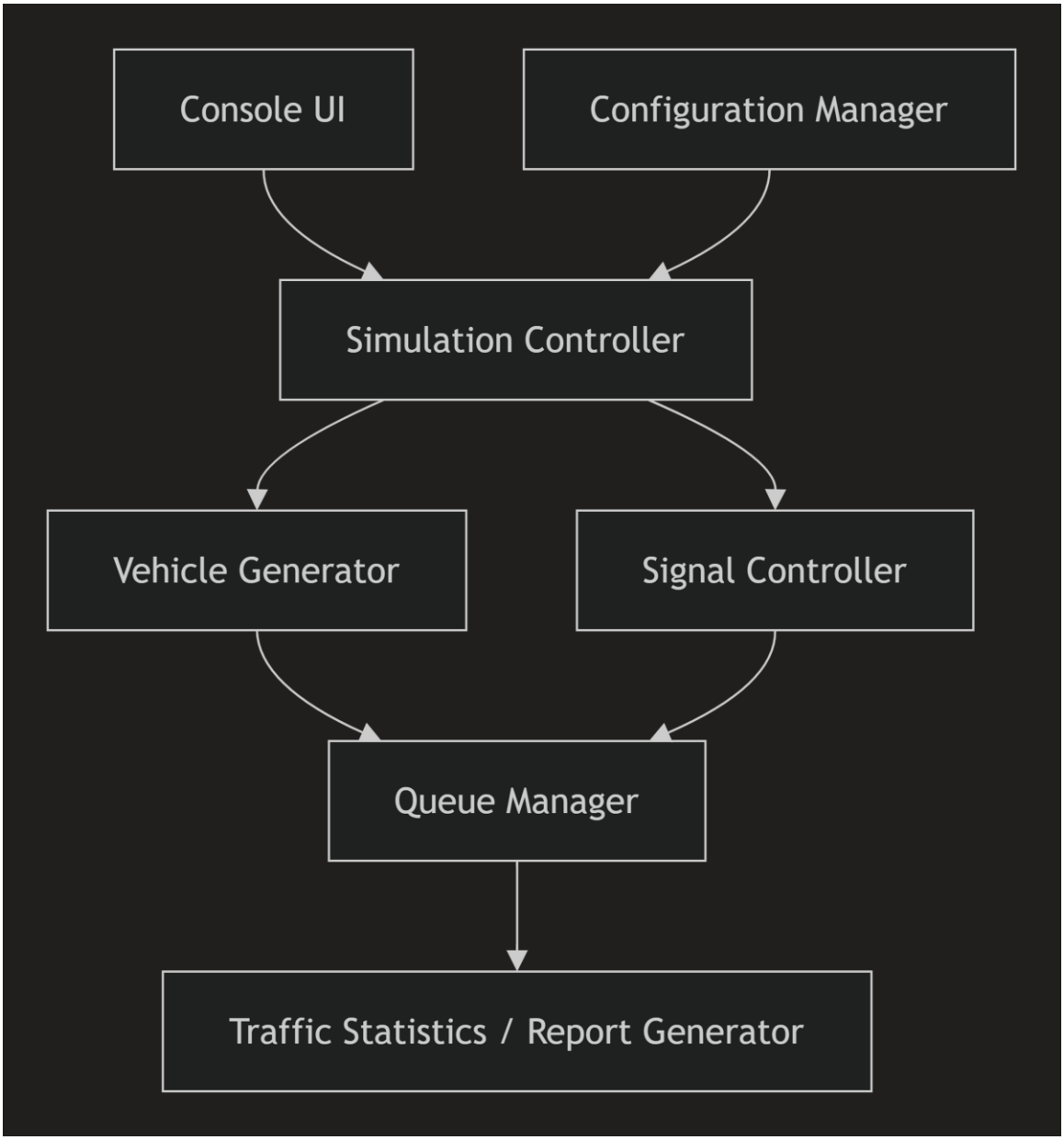
- The system is implemented in C.
- The system is a standalone application.
- No Internet or external server is required.

- No external database is required.
- No specialized traffic hardware is used.
- No physical traffic lights, cameras, road sensors, GPS devices or IoT traffic-control devices are directly connected.
- The system relies on standard C libraries and data structures.

3.2 Stakeholders & Concerns

Stakeholder	**Main Concerns**
Project Developers	Modularity, correctness and maintainability
Project Guide/Faculty	Proper architecture and implementation
Testers	Testability and predictable module behavior
Students	Demonstration of queues and traffic simulation
End Users	Simple console interaction and clear simulation output

3.3 Component (UML) Diagram



Components:

1. **Console UI**

- Accepts user input.
- Displays menus and simulation information.
- Displays signal state, queue lengths and statistics.

2. **Simulation Controller**

- Controls the overall simulation.
- Handles start, pause/resume and stop.
- Coordinates other modules.

3. **Vehicle Generator**
- Generates vehicles according to the configured traffic density.
- Sends generated vehicles to the appropriate queues.

4. **Queue Manager**
- Maintains vehicle queues.
- Performs enqueue and dequeue operations.
- Ensures FIFO behavior.

5. **Signal Controller**
- Maintains Red, Yellow and Green signal states.
- Applies configured signal timings.
- Controls when vehicles can move.

6. **Configuration Manager**
- Stores and validates traffic density and signal timing parameters.

7. **Traffic Statistics / Report Generator**
- Calculates and displays queue lengths, processed vehicles and other traffic statistics.

3.4 Component Descriptions

1. Console UI

Responsibility: Provides the interface between the user and the simulation.

Inputs:

- Menu selections
- Traffic density
- Signal timing
- Start/pause/resume/stop commands

Outputs:

- Signal status
- Queue lengths
- Waiting vehicles
- Passed vehicles
- Traffic statistics
- Error messages

2. Vehicle Generator

Responsibility:

- Generate vehicles.
- Assign vehicles to traffic directions.
- Generate vehicles according to configured traffic density.

3. Queue Manager

Responsibility:

- Maintain queues for different directions.
- Perform enqueue operations.
- Perform dequeue operations.
- Maintain FIFO order.
- Prevent invalid operations such as dequeueing from an empty queue.

4. Signal Controller

Responsibility:

- Maintain signal states.
- Control Red, Yellow and Green transitions.
- Apply configured signal timings.
- Inform the simulation controller whether vehicles can move.

5. Configuration Manager

Responsibility:

- Accept traffic density settings.
- Accept signal timing settings.
- Validate configuration values.
- Provide configuration values to the simulation.

6. Simulation Controller

Responsibility:

- Start the simulation.
- Pause/resume the simulation.
- Stop the simulation.
- Coordinate vehicle generation, queues and signals.

7. Report Generator

Responsibility:

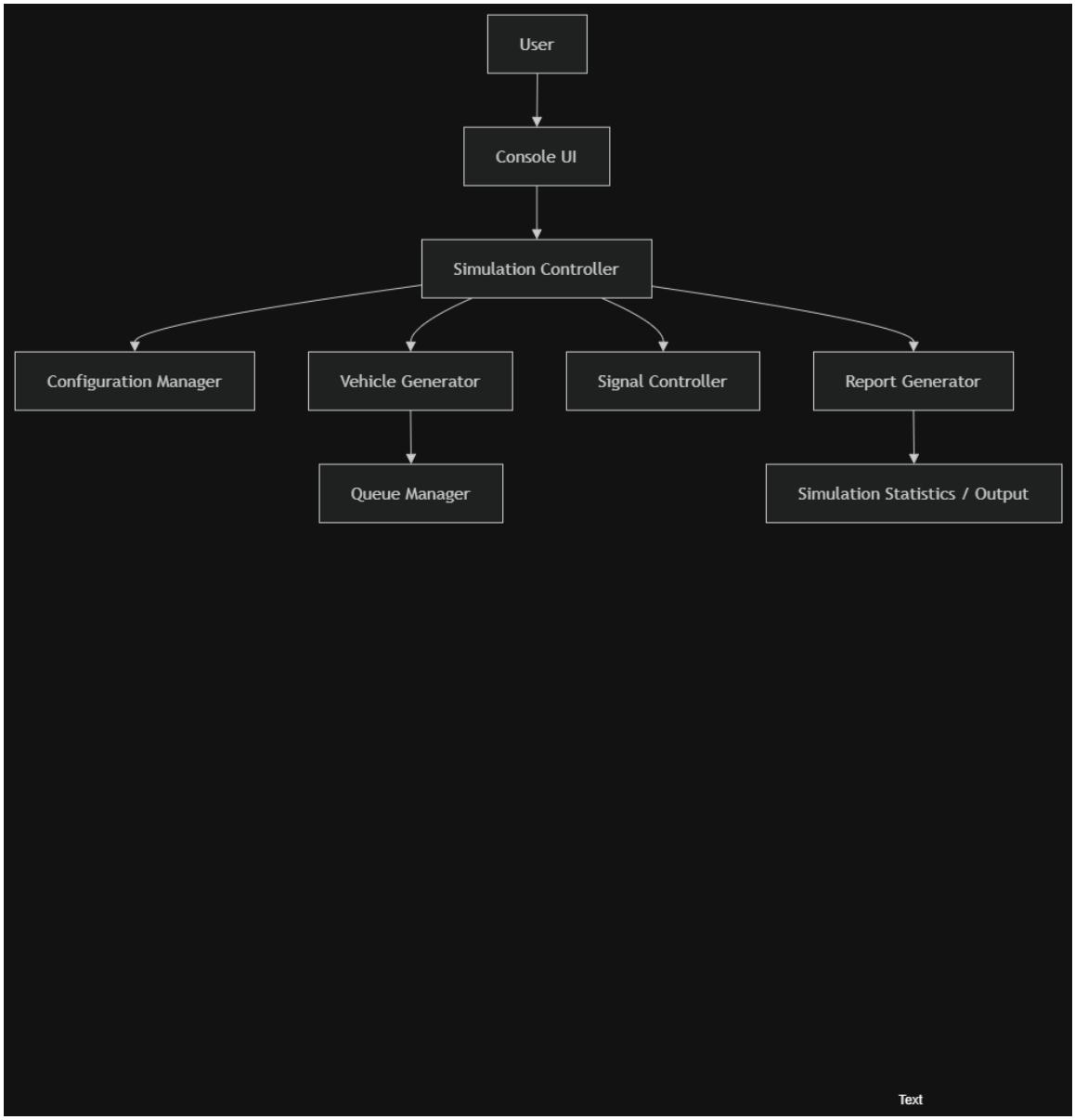
- Calculate traffic statistics.
- Display queue lengths.
- Display number of vehicles processed.
- Display waiting information.
- Present simulation results.

3.5 Chosen Architecture Pattern and Rationale

Chosen Architecture: Layered / Modular Architecture

A layered modular architecture is suitable for the Traffic Management System because the project consists of several independent but interacting responsibilities.

The system can be organized into the following logical layers:



Rationale:

The layered/modular approach is selected because:

- Each module has a clear responsibility.
- Queue operations are separated from signal control.
- Vehicle generation is separated from simulation control.
- Configuration can be validated independently.
- Modules can be tested individually.
- Changes in one module have limited effect on other modules.
- It is simple enough for a C-based educational project.

A microservices or distributed architecture is unnecessary because the SRS specifies that the system is a standalone application with no external communication.

3.6 Technology Stack & Data Stores

Category	**Technology**
Programming Language	C
Compiler	GCC / Standard C Compiler
Operating Systems	Windows, Linux, macOS
Interface	Console / Terminal
Data Structure	Queue
Supporting Structures	Structures, Arrays, Linked Lists where required
Input/Output	stdio.h
Utility/Memory	stdlib.h
Timing/Randomization	time.h
String Operations	string.h, if required
Database	Not required
External Software	Not required

3.7 Risks & Mitigations

Risk	**Mitigation**
Invalid user input	Validate input before processing
Dequeue from empty queue	Check queue status before dequeue
Incorrect queue count	Maintain and verify queue counters

Memory errors	Safely allocate/release memory where dynamic allocation is used
Buffer overflow	Restrict user input size
Invalid signal timing	Validate timing values
Incorrect signal transition	Centralize signal control in Signal Controller
Simulation abnormal termination	Handle unexpected inputs and invalid operations safely

3.8 Traceability to Requirements

SRS Requirement Area	**Design Component**
Vehicle arrival	Vehicle Generator
Separate vehicle queues	Queue Manager
Enqueue operation	Queue Manager
Dequeue operation	Queue Manager
Red/Yellow/Green signals	Signal Controller
Vehicle movement	Simulation Controller + Signal Controller + Queue Manager
Traffic density	Configuration Manager + Vehicle Generator
Signal timing	Configuration Manager + Signal Controller
Queue length	Queue Manager + Report Generator
Traffic statistics	Report Generator
Start/Pause/Resume/Stop	Simulation Controller
Input validation	Console UI + Configuration Manager
Safe queue operations	Queue Manager
Memory safety	Queue Manager / Memory management
Buffer overflow protection	Console UI / Input handling

3.9 Security Architecture

Security Objectives:

SO-001 – Data Integrity

The system shall maintain the accuracy and consistency of:

- Vehicle information
- Queue contents
- Traffic signal states
- Traffic statistics

SO-002 – Safe and Controlled Operation

Only valid user inputs and permitted operations should modify simulation parameters or control the simulation

Security Design:

The design addresses:

- Input validation
- Empty queue protection
- Consistent vehicle counts
- Safe memory management
- Unexpected input handling
- Buffer overflow prevention

4. Design

4.1 Design Overview

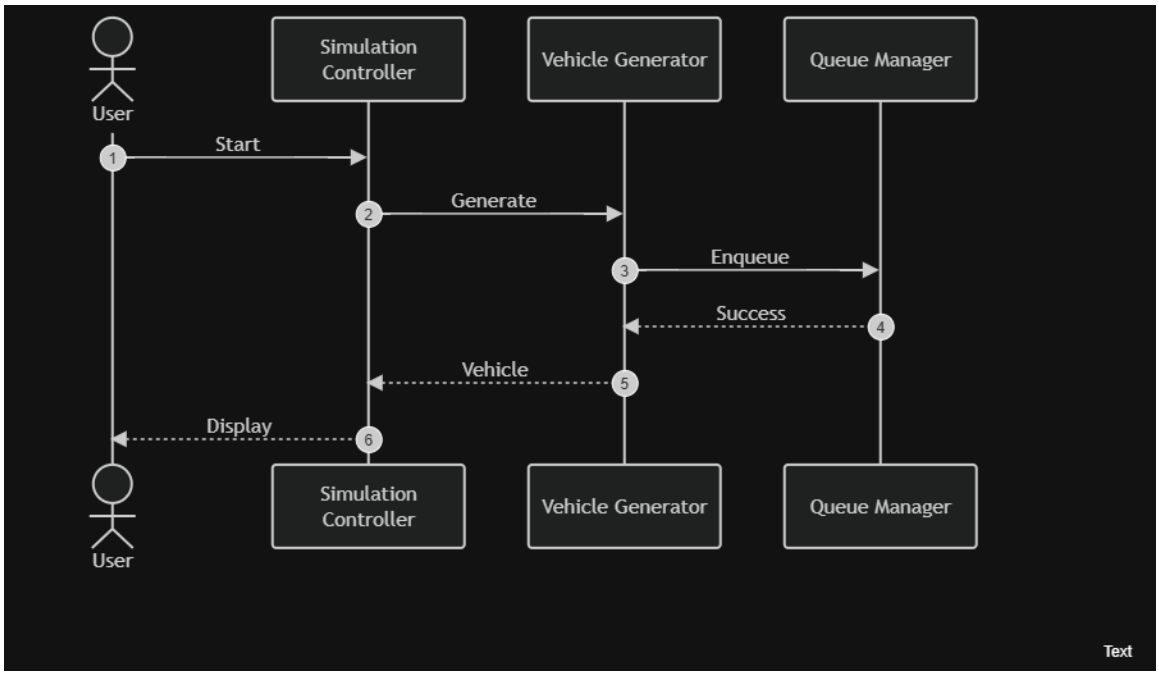
The Traffic Management System follows a modular design where each major operation is handled by a dedicated component.

The general flow is:

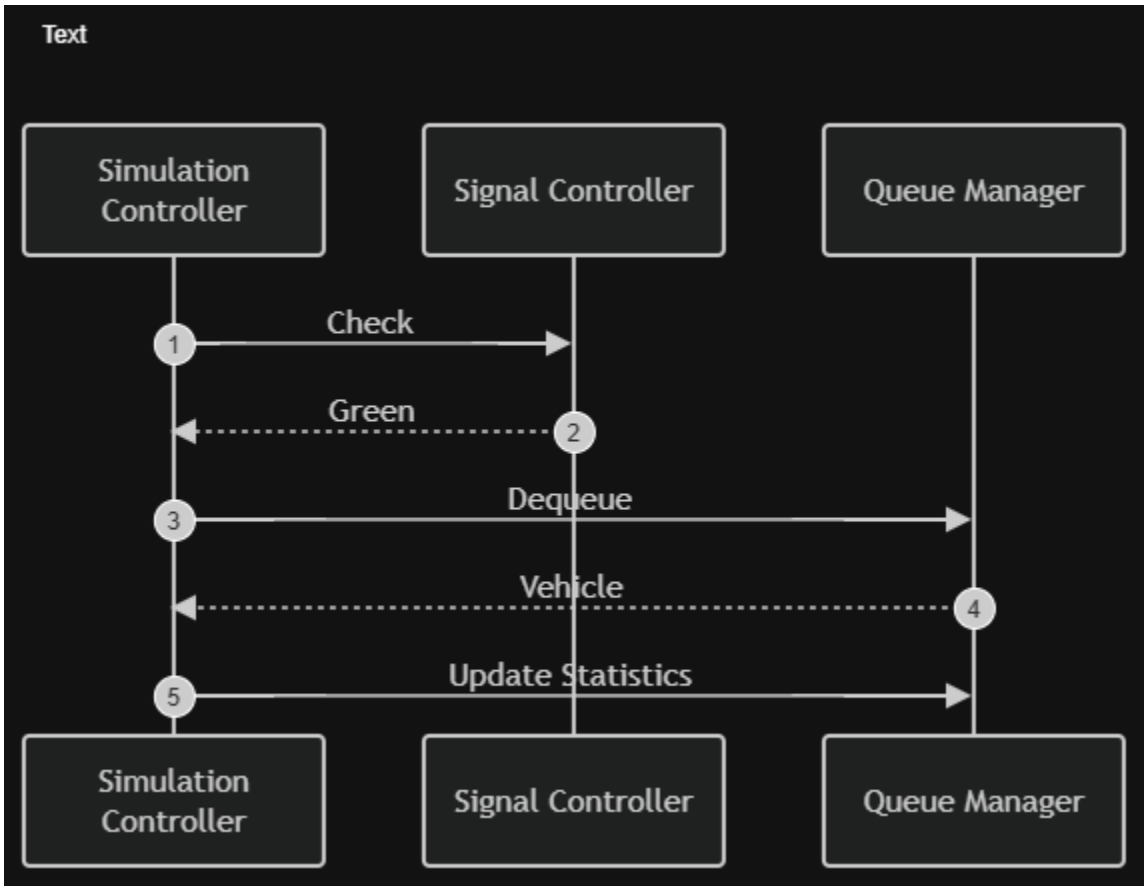
The design ensures that vehicle generation, queue operations, signal control, configuration and reporting remain logically separated.

4.2 UML Sequence Diagrams

Sequence Diagram 1 – Vehicle Arrival and Queue Management :



Sequence Diagram 2 – Signal Change and Vehicle Movement :



4.3 API Design

These interfaces are proposed for implementation and may be adjusted during coding while maintaining the defined responsibilities of each component.

Queue Manager Interface:

void enqueue(Queue *queue, Vehicle vehicle);

Vehicle dequeue(Queue *queue);

int isEmpty(Queue *queue);

int getQueueLength(Queue *queue);

These functions will be used by the Simulation Controller and Signal Controller to manage vehicles according to the FIFO principle.

Signal Controller Interface:

void initializeSignal();

void updateSignal();

SignalState getSignalState();

void setSignalTiming(int red, int yellow, int green);

These functions will be used to initialize the traffic signals, update their states during simulation, retrieve the current signal state, and configure signal timings.

Vehicle Generator Interface:

Vehicle generateVehicle(int direction);

void generateVehicles(int density);

Simulation Controller Interface:

void startSimulation();

void pauseSimulation();

void resumeSimulation();

void stopSimulation();

4.4 Error Handling, Logging & Monitoring

Error Handling:

- Invalid menu options will display an appropriate error message.
- Negative or invalid traffic density values must be rejected.
- Invalid signal timing values must be rejected.
- Attempting to dequeue from an empty queue shall be handled safely without causing a crash

- Excessively long user input must be handled safely.
- Unexpected or invalid input during processing must not cause the program to crash abnormally.

Logging:

The basic Traffic Management System is a standalone application and does not require an external logging server. If logging is implemented, it would log:

- Events such as simulation start/stop
- Traffic signal changes
- Vehicle processing
- Invalid input or errors
- Configuration changes

Monitoring:

The user can monitor the application via the console, including:

- Current traffic signal states
- Number of waiting vehicles
- Queue length for each direction
- Number of vehicles passed through the intersection
- Traffic statistics

4.5 UX Design

The Traffic Management System uses a **console-based user interface** developed in the C programming language. The user interacts with the system through the keyboard and views the simulation output on the screen.

The interface shall provide:

- A main menu for accessing different functions of the system.
- Options to start, pause, resume, and stop the simulation.
- Options to configure traffic density and traffic signal timings.
- Display of the current traffic signal status.
- Display of vehicles waiting in each direction.
- Display of the queue length for each direction.
- Display of vehicles that have passed through the intersection.
- Display of traffic statistics during or after the simulation.
- Clear prompts for entering values.
- Appropriate error messages for invalid inputs.

The user interface is designed to be simple and easy to operate through keyboard input.

4.6 Open Issues & Next Steps

The current version of the Traffic Management System is limited to a standalone traffic simulation. The following features may be considered as future enhancements:

- Integration with real-time traffic information.
- Integration with traffic sensors.
- Integration with traffic cameras.
- Integration with GPS-based traffic information.
- Integration with IoT traffic-control devices.
- Graphical user interface for improved visualization.
- Integration with external databases for storing simulation results.

These features are outside the scope of the current project and may be considered for future extensions.

5. Appendices

5.1 Glossary:

Term	**Definition**
TMS	Traffic Management System
SAD	Software Architecture and Design Specification
SRS	Software Requirements Specification
UML	Unified Modeling Language
FIFO	First-In-First-Out
UI	User Interface
CPU	Central Processing Unit
RAM	Random Access Memory
OS	Operating System
Queue	A data structure used to store waiting vehicles
Enqueue	Operation of adding a vehicle to the rear of a queue
Dequeue	Operation of removing a vehicle from the front of a queue

Signal Controller	Module responsible for controlling traffic signal states
Vehicle Generator	Module responsible for generating simulated vehicles
Simulation Controller	Module responsible for controlling the overall simulation

5.2 References:

1. Traffic Management System – Software Requirements Specification (SRS).
2. Traffic Management System – Software Test Plan.
3. Software Architecture and Design Specification (SAD) Template

5.3 Tools:

The following tools and technologies are used or may be used for the development and documentation of the Traffic Management System:

- **C Programming Language** – implementation of the traffic simulation.
- **GCC** – compilation of the C program.
- **VS Code** – source-code development.
- **draw.io / diagrams.net** – creation of UML diagrams.
- **Microsoft Word** – preparation of project documentation.